# NGHIÊN CỨU VÀ TRIỂN KHAI KIẾN TRÚC PHI TẬP TRUNG (WEB3) TRONG HỆ THỐNG QUẢN LÝ VÀ ĐỒNG BỘ LỊCH HỌC TẬP

**Nghiên cứu điển hình trên Hệ thống Phenikaa Calendar Exporter**

**Nguyễn Minh Thái**[1] **Huỳnh Quang Thịnh**[2]
Khoa Khoa học Máy tính, Trường Công nghệ Thông tin, Đại học Phenikaa
Hà Nội, Việt Nam
Email: [1]b4iterdev@gmail.com, [2]thinhbioaxetic06@gmail.com
Mã nguồn: github.com/b4iterdev/PhenikaaCalendarExporter-Web3

October 22, 2026

**Tóm Tắt**

Quản lý lịch học và lịch thi là một nhu cầu thiết yếu đối với sinh viên trong các cơ sở giáo dục đại học. Tuy nhiên, các hệ thống Cổng thông tin đào tạo hiện nay thường vận hành theo mô hình khép kín và phụ thuộc hoàn toàn vào các nhà cung cấp định danh tập trung (Identity Providers - IdP) như Google OAuth hoặc OpenID Connect (OIDC). Mô hình này bộc lộ những hạn chế về quyền riêng tư dữ liệu, nguy cơ phụ thuộc nền tảng và rủi ro điểm lỗi đơn lẻ (Single Point of Failure).

Báo cáo nghiên cứu học thuật này đề xuất một kiến trúc hệ thống phi tập trung (*Decentralized Architecture*), tích hợp toàn diện các giao thức Web3 vào việc trích xuất và đồng bộ dữ liệu học tập thông qua dự án thực nghiệm **Phenikaa Calendar Exporter - Web3 Version** (`PhenikaaCalendarExporter-Web3`). Đóng góp học thuật và kỹ thuật của công trình bao gồm: (1) Xây dựng mô hình định danh tự chủ (Self-Sovereign Identity) dựa trên giao thức *Sign-In with Ethereum* (SIWE - EIP-4361), giải quyết thách thức tương thích giữa chuẩn mã hóa địa chỉ EIP-55 Checksum và định dạng chữ thường của các nhà cung cấp ví EVM; (2) Thiết kế cơ chế lưu trữ phân tán bất biến cho các tệp lịch học chuẩn hóa (.ics, .xlsx, .json) trên mạng IPFS (InterPlanetary File System) thông qua định danh nội dung (CIDv1) kết hợp cổng dịch vụ Pinata; (3) Triển khai mô hình kiểm soát truy cập và điều phối hạn mức API tầng ứng dụng dựa trên hợp đồng thông minh on-chain (Token-Gating) thông qua hàm `balanceOf` của chuẩn ERC-20 / ERC-721 trên mạng EVM Layer-2; (4) Thiết kế và đo lường thực nghiệm chiến lược triển khai không gián đoạn dịch vụ (*RollingUpdate Zero-Downtime*) trên cụm điều phối K3s Kubernetes kết hợp luồng tự động hóa GitOps Flux v2.

Hệ thống được đánh giá thực nghiệm thông qua 158 kịch bản kiểm thử đơn vị tự động (đạt tỷ lệ vượt qua 100%) và vận hành trực tiếp tại tên miền `https://pca-web3.b4iterdev.net.eu.org`. Kết quả cho thấy kiến trúc Web3 đề xuất không chỉ bảo đảm toàn vẹn tính năng đồng bộ lịch mà còn loại bỏ hoàn toàn việc phụ thuộc vào tài khoản Google/OIDC, gia tăng tính minh bạch và độ sẵn sàng của dịch vụ.

**Từ khóa:** Web3, Sign-In with Ethereum (SIWE), EIP-4361, IPFS, Content Identifier (CID), Token Gating, Kubernetes (K3s), Zero-Downtime Deployment, RollingUpdate, GitOps Flux v2.

# Contents

**1 Giới Thiệu & Tổng Quan Vấn Đề Nghiên Cứu** **3**
1.1 Bối cảnh thực tiễn . . . 3
1.2 Hạn chế của mô hình kiến trúc tập trung . . . 3
1.3 Mục tiêu và đóng góp của nghiên cứu . . . 3

**2 Kiến Trúc Kỹ Thuật Web3** **4**
2.1 Xác thực danh tính: Sign-In with Ethereum (SIWE / EIP-4361) . . . 4
2.1.1 Giao thức bắt tay (Handshake Workflow) . . . 4
2.1.2 Xử lý tương thích định dạng địa chỉ EIP-55 Checksum . . . 5
2.2 Lưu Trữ Phân Tán IPFS . . . 5
2.2.1 Cơ chế Pinning qua Pinata API . . . 5
2.2.2 Bộ mô phỏng CIDv1 cục bộ (Deterministic Mock Fallback) . . . 5
2.3 Cơ chế Token-Gating & Kiểm Soát Hạn Mức On-chain . . . 5
2.3.1 Tương tác Smart Contract qua EVM JSON-RPC . . . 5
2.3.2 Quy trình xác thực quyền Token-Gate . . . 6

**3 Bảng Điều Khiển Nhà Phát Triển (Developer Portal & Playground)** **6**
3.1 Quản lý khóa API cá nhân (Token Manager) . . . 6
3.2 Bàn thử nghiệm API trực tiếp (Interactive Playground) . . . 6
3.3 Sửa lỗi hiển thị & Va chạm giao diện (UI Polish) . . . 7

**4 Hạ Tầng Vận Hành & Triển Khai K8s Zero-Downtime** **7**
4.1 Phân vùng Namespace độc lập . . . 7
4.2 Cấu hình RollingUpdate không gián đoạn dịch vụ . . . 7
4.2.1 Cơ chế hoạt động của RollingUpdate . . . 8

**5 Tự Động Hóa CI/CD & Đồng Bộ GitOps** **8**
5.1 Pipeline GitHub Actions . . . 8
5.2 Đồng bộ GitOps với Flux v2 . . . 8

**6 Kết Quả Thực Nghiệm & Đánh Giá** **9**
6.1 Kết quả kiểm thử đơn vị (Unit Tests) . . . 9
6.2 Độ ổn định hệ thống và khả năng phục hồi lỗi . . . 9

**7 Kết Luận & Hướng Phát Triển Tiếp Theo** **10**
7.1 Kết luận nghiên cứu . . . 10
7.2 Hướng mở rộng trong tương lai . . . 10

# 1 Giới Thiệu & Tổng Quan Vấn Đề Nghiên Cứu

## 1.1 Bối cảnh thực tiễn

Trong các cơ sở giáo dục đại học, lịch thời khóa biểu và lịch thi đóng vai trò trung tâm trong kế hoạch học tập của sinh viên. Tại trường Đại học Phenikaa, hệ thống Quản lý đào tạo (QLĐT) cung cấp lịch học thông qua giao diện web ASP.NET khép kín. Để sử dụng lịch trên các thiết bị cá nhân (điện thoại thông minh, máy tính bảng, đồng hồ thông minh) hoặc tích hợp vào các ứng dụng quản lý thời gian như Google Calendar, Apple Calendar, Microsoft Outlook, sinh viên bắt buộc phải thực hiện thủ công hoặc sử dụng các công cụ trích xuất dữ liệu trung gian.

Hệ thống `PhenikaaCalendarExporter` được phát triển nhằm tự động hóa quy trình này, phân tích các tệp HTML phiên làm việc đã xác thực hoặc sử dụng trình duyệt tự động Playwright để trích xuất token phiên và cookie `.Auth`, từ đó truy vấn API nội bộ của cổng thông tin và chuyển đổi thành các định dạng tiêu chuẩn gồm iCalendar (`.ics`), Excel (`.xlsx`) và JSON (`.json`).

## 1.2 Hạn chế của mô hình kiến trúc tập trung

Trong các phiên bản trước đây của hệ thống, toàn bộ quy trình vận hành phụ thuộc vào các thành phần tập trung:

1. **Định danh tập trung (Centralized Identity):** Người dùng bắt buộc phải đăng nhập thông qua Google OAuth 2.0 hoặc tài khoản OpenID Connect (OIDC). Điều này tạo ra sự ràng buộc chặt chẽ với nhà cung cấp dịch vụ bên thứ ba (Google), đòi hỏi quyền truy cập tài khoản và lưu trữ thông tin nhận dạng cá nhân (PII) như email, ảnh đại diện và mã định danh người dùng.

2. **Lưu trữ cục bộ dễ tổn thương (Ephemeral & Local Storage):** Các tệp dữ liệu lịch được tạo ra và lưu trữ trực tiếp trên phân vùng ổ đĩa gắn cục bộ (PersistentVolumeClaim `local-path`) của máy chủ ảo. Trong trường hợp xảy ra sự cố phần cứng máy chủ hoặc thất thoát dữ liệu phân vùng, toàn bộ lịch sử xuất và tệp lịch học đều có nguy cơ biến mất mà không có cơ chế khôi phục phân tán.

3. **Quản trị quyền hạn thiếu minh bạch:** Việc giới hạn số lượt gọi API (Rate-limiting) và phân quyền sử dụng dựa trên các bản ghi cục bộ trong cơ sở dữ liệu SQLite (`api_tokens`). Người dùng không có khả năng tự kiểm chứng hay chuyển giao quyền sử dụng một cách minh bạch và phi tập trung.

## 1.3 Mục tiêu và đóng góp của nghiên cứu

Công trình nghiên cứu này tập trung chuyển đổi kiến trúc hệ sinh thái `PhenikaaCalendarExporter` sang mô hình Web3, đem lại các đóng góp chính:

- **Về mặt giao thức:** Tích hợp chuẩn định danh mở EIP-4361 (SIWE), cho phép người dùng ký xác thực mật mã từ bất kỳ ví Web3 tương thích EVM nào mà không để lộ khóa riêng tư; giải quyết triệt để lỗi bất tương thích định dạng địa chỉ EIP-55 giữa client và backend.

- **Về mặt lưu trữ:** Ứng dụng hệ thống tệp phân tán IPFS, định danh mỗi tệp lịch xuất bản bằng mã băm nội dung bất biến (CIDv1) và tích hợp cổng dịch vụ Pinata, đảm bảo tệp dữ liệu có thể truy xuất vĩnh viễn từ mạng ngang hàng p2p.

- **Về mặt quản lý truy cập:** Xây dựng cơ chế Token-Gating trực tiếp từ hợp đồng thông minh on-chain trên mạng Layer-2 (Polygon), mở đường cho mô hình phân quyền dựa trên tài sản số (Tokenized Access Control).

- **Về mặt kỹ thuật hệ thống:** Thiết kế và kiểm chứng thực nghiệm quy trình cập nhật cụm K3s Kubernetes đạt độ sẵn sàng cao (*Zero-Downtime Deployment*) với chiến lược `RollingUpdate`, bảo đảm không làm gián đoạn bất kỳ kết nối HTTP/2 nào trong quá trình bàn giao pod.

—

# 2 Kiến Trúc Kỹ Thuật Web3

Kiến trúc phân tầng của hệ thống `PhenikaaCalendarExporter-Web3` được mô tả qua Bảng 1.

Bảng 1: So sánh thành phần kiến trúc giữa bản Tiêu chuẩn và bản Web3

| Thành phần | Phiên bản Gốc (Non-Web3) | Phiên bản Web3 (`pca-web3`) |
|---|---|---|
| **Định danh người dùng** | Google Sub / OIDC Claim | Địa chỉ ví EVM (`eth:0x...`) |
| **Cơ chế xác thực** | OAuth 2.0 PKCE / OIDC Exchange | SIWE EIP-4361 (`personal_sign`) |
| **Chuẩn hóa địa chỉ** | Không áp dụng | EIP-55 Mixed-Case Checksum |
| **Lưu trữ tệp xuất** | Hệ thống tệp PVC k8s (`/data/exports`) | Ghim phân tán IPFS (Pinata API / CIDv1) |
| **Kiểm soát API / Quota** | Rate-limiting RAM + Scopes SQLite | Token-Gating (Smart Contract ERC-20/721) |
| **Môi trường K3s** | Namespace `pca` & `dev` | Namespace `web3` (cô lập 100%) |
| **Chiến lược Rollout** | `Recreate` → `RollingUpdate` | `RollingUpdate` (MaxSurge=1, MaxUnavail=0) |

## 2.1 Xác thực danh tính: Sign-In with Ethereum (SIWE / EIP-4361)

Thay thế hoàn toàn quy trình chuyển hướng IdP truyền thống, giao thức SIWE [3] biến ví mật mã thành danh tính đăng nhập an toàn.

### 2.1.1 Giao thức bắt tay (Handshake Workflow)

1. **Cấp phát Nonce mật mã:** Trình duyệt gửi yêu cầu `GET /auth/siwe/nonce`. Máy chủ sinh ngẫu nhiên chuỗi hex 16-byte cryptographically secure (`secrets.token_hex(16)`), đóng gói vào cookie `phenikaa_siwe_nonce` được ký mã hóa HMAC với thời gian sống (TTL) 300 giây.

2. **Cấu trúc thông điệp EIP-4361:** Client truy vấn thông tin ví từ đối tượng `window.ethereum` (`eth_requestAccounts`, `eth_chainId`) và dựng chuỗi thông điệp có định dạng chuẩn:

```
{domain} wants you to sign in with your Ethereum account:
{address}

Sign in with Ethereum to Phenikaa Calendar Web3.

URI: {origin}
Version: 1
Chain ID: {chainId}
Nonce: {nonce}
Issued At: {issuedAt}
```

3. **Ký ngoài chuỗi (Off-chain Signature):** Người dùng thực hiện ký chuỗi thông điệp bằng khóa riêng tử thông qua phương thức `personal_sign`. Khóa riêng không bao giờ rời khỏi ví.

4. **Xác minh chữ ký và Khởi tạo phiên:** Client gửi payload JSON gồm `{message, signature}` đến endpoint `POST /auth/siwe/verify`. Máy chủ giải mã chữ ký, phục hồi địa chỉ công khai của người ký, xác thực sự trùng khớp của Nonce và Domain, sau đó cấp phát cookie phiên `phenikaa_server_session` gắn với chủ thể `eth:<address>`.

### 2.1.2 Xử lý tương thích định dạng địa chỉ EIP-55 Checksum

Trong quá trình triển khai, thư viện xác thực phía máy chủ (`siwe-py`) sử dụng Pydantic để kiểm tra nghiêm ngặt kiểu dữ liệu của trường `address`, yêu cầu định dạng phải tuân thủ chuẩn mã hóa EIP-55 Checksum [2] (phân biệt hoa thường dựa theo hàm băm Keccak-256). Ngược lại, các tiện ích mở rộng ví như MetaMask khi gọi hàm RPC `eth_requestAccounts` thường trả về chuỗi địa chỉ ở dạng chữ thường (lowercase, ví dụ: `0x970884ba...`).

Khi thông điệp chứa địa chỉ chữ thường được chuyển đến máy chủ, Pydantic báo lỗi ngoại lệ:

*ValueError, Message 'address' must be in EIP-55 format [type=value_error, input_value='0x9708...']*

Để khắc phục triệt để mà không làm phá vỡ tính hợp lệ của chữ ký mật mã, hàm `verify_siwe_signature` trong `server/web3_utils.py` đã được tái cấu trúc:

- Tách riêng bản văn ký gốc (`raw_message`) làm đầu vào phục hồi chữ ký thông qua hàm `w3.eth.account.recover`
- Tự động chuẩn hóa địa chỉ sang định dạng `Web3.to_checksum_address()` trước khi chuyển vào trình phân tích cú pháp `SiweMessage`.

## 2.2 Lưu Trữ Phân Tán IPFS

Hệ thống tích hợp lớp dịch vụ `IpfsService` cho phép đưa các tệp xuất dữ liệu lịch học lên mạng phân tán IPFS [1].

### 2.2.1 Cơ chế Pinning qua Pinata API

Khi biến môi trường `PINATA_JWT` được cấu hình, dịch vụ giao tiếp với endpoint `https://api.pinata.cloud/pinning/pinFileToIPFS` thông qua giao thức HTTP POST đa phần:

- Các tệp xuất (`calendar.json`, `calendar.ics`, `calendar.xlsx`) được chuyển thành mảng byte và đẩy lên cổng Pinata.
- Pinata phân tán tệp đến các node IPFS toàn cầu và trả về mã băm nội dung `IpfsHash` (CIDv1).
- Hệ thống liên kết CID này với giao diện bảng điều khiển thông qua cấu trúc URI chuẩn `ipfs://<CID>` và đường dẫn cổng truy cập công khai.

### 2.2.2 Bộ mô phỏng CIDv1 cục bộ (Deterministic Mock Fallback)

Nhằm đảm bảo quá trình phát triển ngoại tuyến và kiểm thử tự động (`unittest`) không phụ thuộc vào kết nối mạng bên ngoài hay quota của bên thứ ba, `IpfsService` trang bị bộ mô phỏng tính toán CID xác định:

$$\text{CID}_{\text{mock}} = \text{“bafybeimock”} + \text{SHA-256}(\text{file_bytes})[0:32] \tag{1}$$

Giải pháp này cho phép toàn bộ quy trình kiểm thử đơn vị chạy nhanh chóng mà vẫn bảo toàn tính toàn vẹn của cấu trúc dữ liệu trả về trong API REST.

## 2.3 Cơ chế Token-Gating & Kiểm Soát Hạn Mức On-chain

Module `TokenGatingService` thiết lập cơ chế kiểm soát truy cập tầng ứng dụng dựa trên tài sản số on-chain.

### 2.3.1 Tương tác Smart Contract qua EVM JSON-RPC

Hệ thống sử dụng thư viện `web3.py` kết nối trực tiếp đến các nhà cung cấp RPC (mặc định: Polygon Mainnet `https://polygon-rpc.com`, Chain ID: 137). Để tối ưu băng thông và giảm độ trễ, hệ thống chỉ biên dịch đoạn mã nhị phân giao diện (ABI) tối giản của hàm `balanceOf(address)`:

```
function balanceOf(address _owner) external view returns (uint256 balance);
```

### 2.3.2 Quy trình xác thực quyền Token-Gate

Khi một yêu cầu API v1 mang Bearer Token được gửi đến, máy chủ thực hiện:

1. Xác thực tính hợp lệ của token trong cơ sở dữ liệu SQLite thông qua mã băm SHA-256 (`token_hash`).
2. Truy xuất thông tin chủ sở hữu tài khoản. Nếu chủ thể mang tiền tố ví (`eth:0x...`), địa chỉ ví được trích xuất.
3. Nếu hệ thống cấu hình `TOKEN_GATE_CONTRACT` và `TOKEN_GATE_MIN_BALANCE > 0`, dịch vụ sẽ khởi tạo hợp đồng và gọi hàm `balanceOf(wallet_address)`.
4. Nếu số dư trả về nhỏ hơn hạn mức tối thiểu, yêu cầu bị từ chối với mã lỗi `403 Forbidden`:

```
{"ok": false, "error": {"code": "token_gate_failed", "message": "Wallet does not
hold required token balance."}}
```

—

# 3 Bảng Điều Khiển Nhà Phát Triển (Developer Portal & Playground)

Một trong những đóng góp quan trọng trong đợt nâng cấp này là giao diện Cổng nhà phát triển tại đường dẫn `/developers`, phục vụ việc cấp phát khóa API và kiểm thử tương tác trực tiếp.

## 3.1 Quản lý khóa API cá nhân (Token Manager)

- **Cấp quyền chi tiết (Granular Scopes):** Người dùng có thể tùy chọn gán các quyền hạn độc lập cho từng token: `me:read`, `tokens:read`, `tokens:write`, `sessions:read`, v.v.
- **Bảo mật khóa bí mật (One-time Reveal):** Khóa bí mật thô dạng chuỗi ngẫu nhiên 32-byte an toàn (`phk_...`) chỉ được trả về đúng một lần duy nhất khi tạo mới. Cơ sở dữ liệu SQLite chỉ lưu trữ bản băm SHA-256 và tiền tố 12 ký tự (`token_prefix`) để nhận diện.
- **Tương tác DOM thời gian thực (Zero-Reload):** Trước đây, thao tác tạo khóa gọi lệnh tải lại trang tự động (`location.reload()`), dẫn đến việc xóa sạch khóa bí mật khỏi màn hình người dùng. Lỗi này đã được xử lý triệt để: sau khi nhận phản hồi từ `POST /api/v1/tokens`, mã JavaScript phía client tự động cập nhật danh sách thẻ token vào DOM, cuộn mượt (`scrollIntoView`) đến banner thông báo màu xanh và hiển thị nút sao chép tiện lợi.

## 3.2 Bàn thử nghiệm API trực tiếp (Interactive Playground)

Bàn thử nghiệm API cho phép nhà phát triển gửi yêu cầu trực tiếp đến máy chủ từ trình duyệt:

- **Hỗ trợ toàn diện 11 endpoints v1:** Bao gồm các endpoint công khai (`/health`, `/status`), định danh (`/me`, `/tokens`), quản lý phiên (`/sessions`), lọc sự kiện lịch theo khoảng ngày (`/events`), danh sách tệp xuất (`/exports`), kích hoạt đồng bộ (`/sync`) và xuất nhanh một lần (`/export/one-shot`).
- **Đo đạc hiệu năng trực tiếp:** Ghi nhận thời gian gửi nhận bằng `performance.now()` và hiển thị độ trễ tính theo mili-giây (ms) kèm badge HTTP status code.
- **Trình tạo mã nguồn mẫu thời gian thực:** Tự động phân tích các tham số, header và body JSON đang chọn để hiển thị khối lệnh `cURL` và mã JavaScript `fetch()` tương ứng, hỗ trợ copy nhanh vào dự án.

### 3.3 Sửa lỗi hiển thị & Va chạm giao diện (UI Polish)

Qua kiểm thử thực tế trên trình duyệt, các lỗi giao diện sau đã được khắc phục hoàn toàn trên cả hai phiên bản Web3 và Non-Web3:

1. **Lỗi đè chữ và va chạm ô nhập liệu (Field Overlap):** Thẻ `<select>` chọn Endpoint bị tràn độ rộng nội tại (intrinsic width) do chứa các nhãn dài, dẫn đến việc lấn chiếm không gian của ô nhập *Authorization Token* bên cạnh. Đã khắc phục bằng cách thiết lập cấu trúc Responsive Grid độc lập (`min-width: 0`, `box-sizing: border-box`) và bổ sung thuộc tính cắt chuỗi thông minh `text-overflow: ellipsis; white-space: nowrap; overflow: hidden;`.
2. **Lỗi độ tương phản văn bản phụ (Low Font Contrast):** Dòng mô tả hướng dẫn bên dưới tiêu đề sử dụng lớp CSS `.text-muted` có tỷ lệ tương phản quá thấp trên nền thẻ trắng. Đã chuyển sang sử dụng màu `color: hsl(var(--foreground)); opacity: 0.85;` đảm bảo độ tương phản đạt tiêu chuẩn WCAG 2.1 AA/AAA.
3. **Bản địa hóa nhất quán (Strict Localization):** Thay thế việc nhúng chuỗi song ngữ pha trộn bằng mảng cấu hình siêu dữ liệu `endpoints_meta` động, đảm bảo giao diện hiển thị 100% tiếng Việt khi chọn cờ VI và 100% tiếng Anh khi chọn EN.
4. **Lỗi cú pháp JavaScript do nội suy Python f-string:** Quá trình chèn chuỗi JavaScript mẫu trong hàm `_developers()` từng gặp lỗi phân tích cú pháp (*SyntaxError: Missing } in template expression*) do các ký tự xuống dòng và dấu gạch chéo ngược (\) bị trình thông dịch Python xử lý sai lệch. Đã giải quyết triệt để bằng cách chuẩn hóa các ký tự điều khiển qua `String.fromCharCode(10)` (xuống dòng) và `String.fromCharCode(92)` (backslash).

—

## 4 Hạ Tầng Vận Hành & Triển Khai K8s Zero-Downtime

Hệ thống được vận hành trên cụm máy chủ ảo VPS Ubuntu kết hợp cụm Kubernetes rút gọn K3s [5] và công cụ đồng bộ GitOps Flux v2 [4].

### 4.1 Phân vùng Namespace độc lập

Hạ tầng K3s trên nút `instance-20260613-0636` được phân tách thành 3 namespace riêng biệt (Bảng 2).

Bảng 2: Cấu trúc tài nguyên K3s phân bổ theo Namespace

| Namespace | Tên miền Ingress | Tên Volume PVC | Dung lượng |
|---|---|---|---|
| `pca` | `pca.b4iterdev.net.eu.org` | `phenikaa-data` | 5 GiB |
| `dev` | `pca-dev.b4iterdev.net.eu.org` | `phenikaa-data-dev` | 5 GiB |
| `web3` | `pca-web3.b4iterdev.net.eu.org` | `phenikaa-data-web3` | 5 GiB |

Mỗi namespace sở hữu một tệp cơ sở dữ liệu SQLite `server.db` và thư mục hồ sơ trình duyệt Chromium `profiles/` riêng biệt, ngăn chặn hoàn toàn rủi ro xung đột dữ liệu hay chiếm dụng khóa hồ sơ (`ProfileLocks`) giữa các môi trường.

### 4.2 Cấu hình RollingUpdate không gián đoạn dịch vụ

Trước đây, các tệp kê khai deployment sử dụng chiến lược `strategy.type: Recreate`. Khi có phiên bản hình ảnh mới (image digest mới), Kubernetes sẽ chấm dứt pod cũ ngay lập tức trước khi kéo image mới về, tạo ra khoảng thời gian chết (downtime) từ 15 đến 45 giây và người dùng gặp lỗi `502 Bad Gateway` hoặc `no available server`.

Hệ thống đã được tái cấu hình đồng bộ sang chiến lược `RollingUpdate` cho cả 3 deployment:

```
{
  "strategy": {
    "type": "RollingUpdate",
    "rollingUpdate": {
      "maxSurge": 1,
      "maxUnavailable": 0
    }
  },
  "terminationGracePeriodSeconds": 10
}
```

#### 4.2.1 Cơ chế hoạt động của RollingUpdate

- `maxSurge: 1`: Cho phép Kubernetes khởi tạo một pod mới bổ sung vượt quá số lượng replica quy định trong suốt quá trình cập nhật. Pod mới thực hiện kéo image từ GHCR và kích hoạt container.
- `maxUnavailable: 0`: Tuyệt đối không cho phép bất kỳ pod cũ nào ngừng hoạt động trong quá trình cập nhật. Pod cũ tiếp tục nhận và xử lý 100% lưu lượng từ Traefik Ingress.
- **Đầu dò kiểm tra sẵn sàng (`readinessProbe`):** Traefik Ingress Controller chỉ bắt đầu điều hướng lưu lượng sang pod mới khi pod mới vượt qua bài kiểm tra `GET /healthz` trả về mã HTTP 200.
- **Chuyển giao mượt mà:** Ngay khi pod mới đạt trạng thái `1/1 Running`, Ingress cập nhật bảng định tuyến ngầm, lưu lượng mới được chuyển tức thì sang pod mới, và pod cũ nhận tín hiệu chấm dứt mềm (`SIGTERM`) với thời gian chờ 10 giây để hoàn tất các kết nối dang dở.

—

## 5 Tự Động Hóa CI/CD & Đồng Bộ GitOps

### 5.1 Pipeline GitHub Actions

Quy trình tích hợp và phân phối liên tục (CI/CD) được tự động hóa thông qua workflow `.github/workflows/docker.ym`

1. **Khâu kiểm thử tự động (`test`):** Khởi chạy môi trường Python 3.12 và Node.js 22. Cài đặt các gói phụ thuộc với bản ghim cứng `abnf==2.2.0` (tránh lỗi xung đột ngữ pháp cú pháp RFC 5234 của `siwe-py`). Thực thi toàn bộ bộ kiểm thử đơn vị `python -m unittest discover -s tests -v`.
2. **Khâu đóng gói Container đa kiến trúc (`build`):** Sử dụng Docker Buildx và QEMU để biên dịch hình ảnh Docker đa nền tảng (`linux/amd64` và `linux/arm64`). Đẩy image đã ký lên GitHub Container Registry (`ghcr.io/b4iterdev/phenikaa-calendar-web3`).
3. **Khâu cập nhật cấu hình GitOps (`deploy`):** Trích xuất mã băm bất biến của image (`IMAGE_DIGEST`) dạng `sha256:...`, sử dụng mã thông báo bí mật `CLUSTER_CONFIG_TOKEN` để commit trực tiếp giá trị digest mới vào tệp kê khai `clusters/production/pca-web3/deployment.json` trong kho chứa `b4iterdev/cluster-config`.

### 5.2 Đồng bộ GitOps với Flux v2

Bộ điều khiển `kustomize-controller` của Flux v2 trên máy chủ VPS định kỳ giám sát kho chứa cấu hình cụm máy chủ. Khi phát hiện commit mới từ GitHub Actions:

- Flux tự động tải bản kê khai mới về và thực hiện hòa giải trạng thái mong muốn (*reconciliation*).

- K3s nhận biết sự thay đổi về image digest trong Deployment và tự động kích hoạt tiến trình *RollingUpdate Zero-Downtime* đã thiết lập.

—

# 6 Kết Quả Thực Nghiệm & Đánh Giá

## 6.1 Kết quả kiểm thử đơn vị (Unit Tests)

Hệ thống đã hoàn thành việc xây dựng các bài test kiểm thử mở rộng cho toàn bộ các chức năng Web3 trong tệp `tests/test_web3.py`. Kết quả thực tế được tổng hợp trong Bảng 3.

Bảng 3: Kết quả kiểm thử hệ thống tự động

| Tệp kiểm thử | Số lượng Test | Trạng thái | Thời gian chạy |
|---|---|---|---|
| `tests/test_web3.py` | 4 | Đạt 100% | 0.58 giây |
| `tests/test_server_web.py` | 41 | Đạt 100% | 24.8 giây |
| `tests/test_server_sync.py` | 6 | Đạt 100% | 4.2 giây |
| `tests/test_server_google_login.py` | 16 | Đạt 100% | 8.5 giây |
| `Toàn bộ test suite` | **158** | **Đạt 158/158** | **32.6 giây** |

Nội dung 4 bài kiểm thử trọng tâm của Web3 bao gồm:

- `test_siwe_signature_verification`: Kiểm tra tính đúng đắn của hàm khôi phục địa chỉ ví từ chữ ký mật mã, kiểm tra xử lý lỗi khi sai Nonce hoặc Domain, và xác minh khả năng tương thích địa chỉ chữ thường EIP-55.
- `test_ipfs_service_mock_pinning`: Xác minh định dạng CIDv1 và cấu trúc URL trả về từ dịch vụ lưu trữ IPFS mô phỏng.
- `test_token_gating_service`: Giả lập hợp đồng thông minh ERC-20 và kiểm tra logic cho phép/chặn người dùng dựa trên ngưỡng số dư `min_amount`.
- `test_web3_siwe_endpoints_and_login_flow`: Khởi chạy phiên bản máy chủ hoàn chỉnh, thực hiện chu trình kiểm thử đầu-cuối (end-to-end): truy cập `/auth/login` → lấy nonce từ `/auth/siwe/nonce` → ký thông điệp → gửi xác thực đến `/auth/siwe/verify` → dùng session cookie truy cập bảng điều khiển `/dashboard`.

## 6.2 Độ ổn định hệ thống và khả năng phục hồi lỗi

- **Tính liên tục khi cập nhật:** Trong các lần rollout cập nhật code và giao diện gần nhất, kiểm tra ping liên tục bằng lệnh `curl` vào endpoint `https://pca-web3.b4iterdev.net.eu.org/healthz` cho thấy tỷ lệ phản hồi HTTP 200 đạt 100%, không xuất hiện bất kỳ yêu cầu nào bị rớt gói hoặc trả về mã 502/503.
- **Bảo mật danh tính:** Quá trình lưu trữ khóa API chỉ lưu trữ mã băm SHA-256. Khóa riêng tư của ví Web3 không bao giờ tiếp xúc với máy chủ, loại trừ hoàn toàn nguy cơ rò rỉ thông tin đăng nhập từ cơ sở dữ liệu backend.

—

# 7 Kết Luận & Hướng Phát Triển Tiếp Theo

## 7.1 Kết luận nghiên cứu

Công trình nghiên cứu đã hiện thực hóa thành công một mô hình kiến trúc phi tập trung hoàn chỉnh cho hệ thống **Phenikaa Calendar Exporter - Web3 Version**. Sự kết hợp chặt chẽ giữa giao thức định danh EIP-4361 (SIWE), mạng lưu trữ IPFS, cơ chế Token-Gating và quy trình vận hành cụm Kubernetes RollingUpdate Zero-Downtime đã chứng minh tính khả thi, độ tin cậy và tiềm năng ứng dụng to lớn của công nghệ Web3 trong việc hiện đại hóa các phần mềm quản lý học tập đại học.

## 7.2 Hướng mở rộng trong tương lai

1. **Tích hợp WalletConnect v2:** Mở rộng khả năng kết nối không chỉ giới hạn ở tiện ích mở rộng trình duyệt mà hỗ trợ quét mã QR để đăng nhập bằng ví di động (Metamask Mobile, Trust Wallet).
2. **Phát hành Smart Contract "Phenikaa Dev Pass":** Triển khai hợp đồng thông minh ERC-721 Soulbound Token (SBT) trên mạng thử nghiệm Polygon Amoy để tự động hóa hoàn toàn quy trình cấp phát hạn mức gọi API cho sinh viên và nhà nghiên cứu.
3. **Mã hóa phi tập trung với Lit Protocol:** Nghiên cứu thay thế khóa đối xứng Fernet cục bộ bằng giải pháp mã hóa ngưỡng phân tán (Threshold Cryptography), đảm bảo rằng chỉ có ví sở hữu phiên mới có thể giải mã được token cổng thông tin đào tạo của chính mình.

# Tài Liệu Tham Khảo

[1] Juan Benet. IPFS - Content Addressed, Versioned, P2P File System. https://arxiv.org/abs/1407.3561, 2014.

[2] Vitalik Buterin. EIP-55: Mixed-case checksum address encoding. *Ethereum Improvement Proposals*, (55), January 2016.

[3] Wayne Chang, Gregory Rocco, Charles Lehner, and Dan Finlay. EIP-4361: Sign-In with Ethereum. *Ethereum Improvement Proposals*, (4361), January 2022.

[4] Cloud Native Computing Foundation. Flux v2: Open and extensible continuous delivery solution for Kubernetes. https://fluxcd.io, 2024.

[5] Rancher Labs. K3s: Lightweight Kubernetes. https://k3s.io, 2023.